Projeto 8 – Modulação AM
Rádio com 3 estações

Camada Física da Computação – Insper

1. Visão geral

A entrega tem dois scripts Python, no mesmo formato do Projeto 7:

• **projeto8_am.py** – o Projeto 8: grava 3 áudios, modula cada um em AM numa faixa diferente (9–12, 12–15 e 15–18 kHz), soma os 3 (sinal no meio físico), separa cada estação com filtro passa-faixa, demodula e salva *demodulado1/2/3.wav*. Mostra as FFTs pedidas e toca os áudios originais e demodulados.

• **exercicios.py** – resolve os exercícios 1 a 7 do enunciado. Reaproveita as funções do projeto8_am.py (FFT, filtros, carregar/tocar áudio), então não há código repetido.

Dependências: numpy, scipy, matplotlib e sounddevice (as mesmas do Projeto 7).
Para rodar (dentro da pasta Projeto 8): `python projeto8_am.py` ou `python exercicios.py` ou `python projeto8_am.py teste` (autoteste sem microfone).

Na primeira execução, como *audio1.wav*, *audio2.wav* e *audio3.wav* ainda não existem, o programa pede para apertar Enter e grava 4 s pelo microfone para cada um. Nas próximas execuções usa os arquivos salvos (dá para trocar por qualquer .wav de 44100 Hz).

Fluxo do programa

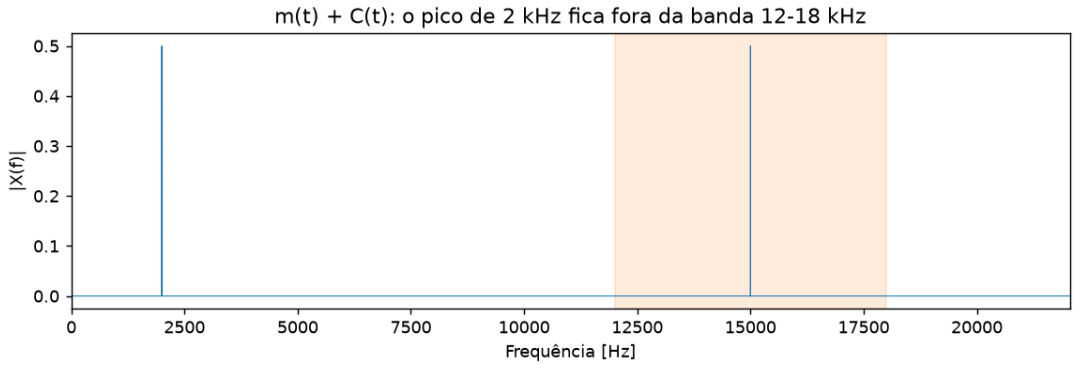
Lado	Etapa	Função
Emissor	Grava/lê os 3 áudios e toca os originais	carrega_audio, toca
Emissor	Passa-baixa de 1,2 kHz em cada áudio (limita a banda)	transmite → passa_baixa
Emissor	Multiplica pela portadora do centro da faixa (10,5 / 13,5 / 16,5 kHz)	transmite
Meio físico	Soma das 3 estações = sinal que o receptor capta	transmite
Receptor	Passa-faixa em cada faixa (sintoniza a estação)	recebe → passa_faixa
Receptor	Multiplica pela portadora de novo + passa-baixa de 1,2 kHz	recebe → passa_baixa
Receptor	Salva demodulado1/2/3.wav, mostra FFTs e toca	salva, plota_fft, toca

2. Exercícios (exercicios.py)

Exercício 1 – Somar não é modular

Geramos 1 s a 44100 Hz de uma senoide de 2 kHz (informação) e outra de 15 kHz (portadora) e somamos. A FFT da soma tem dois picos, um em 2 kHz e outro em 15 kHz: a soma só junta os espectros dos dois sinais. O pico de 2 kHz está fora da banda permitida (12–18 kHz, sombreada), então esse sinal não pode ser transmitido.

```
m = np.sin(2 * np.pi * 2000 * t)                    # informação
c = np.sin(2 * np.pi * 15000 * t)                   # portadora
plota_fft([m + c], [...], "Exercício 1", faixas=[(12000, 18000)])
```



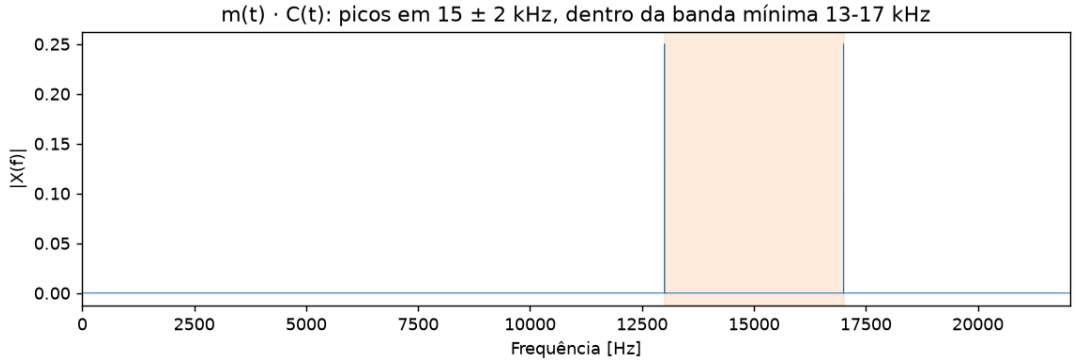
Exercício 2 – Banda mínima

Pela identidade trigonométrica do enunciado, o produto C(t)·m(t) só tem as frequências $f_c - f_m$ e $f_c + f_m$. Com f_c = 15 kHz e f_m = 2 kHz: **banda mínima de 13 kHz a 17 kHz** (largura $2 \cdot f_m$ = 4 kHz).

Exercício 3 – Modulação por multiplicação

Mesmos sinais, agora com 2 s, multiplicados ponto a ponto (em numpy, a `*` b entre dois arrays já multiplica elemento a elemento). Como sen(a)·sen(b) = [cos(a–b) – cos(a+b)]/2, a FFT mostra só dois picos, em 13 kHz e 17 kHz, ambos dentro da banda mínima do Exercício 2. Não sobra nada em 2 kHz nem em 15 kHz.

```
t = np.arange(0, 2, 1 / FS)
s = np.sin(2 * np.pi * 2000 * t) * np.sin(2 * np.pi * 15000 * t)
```



Exercício 4 – Informação de 20 Hz a 2 kHz na banda de 14 a 16 kHz

Com portadora de 15 kHz, o sinal modulado ocupa 15 kHz ± $f_{m,máx}$. Para caber em 14–16 kHz, a informação só pode ter até 1 kHz. Então, antes de modular, passamos m(t) num **filtro passa-baixa de 1 kHz**. Perdemos o conteúdo de 1 a 2 kHz (o som fica mais abafado), mas o sinal modulado respeita a banda. É exatamente o que o Projeto 8 faz com os áudios gravados.

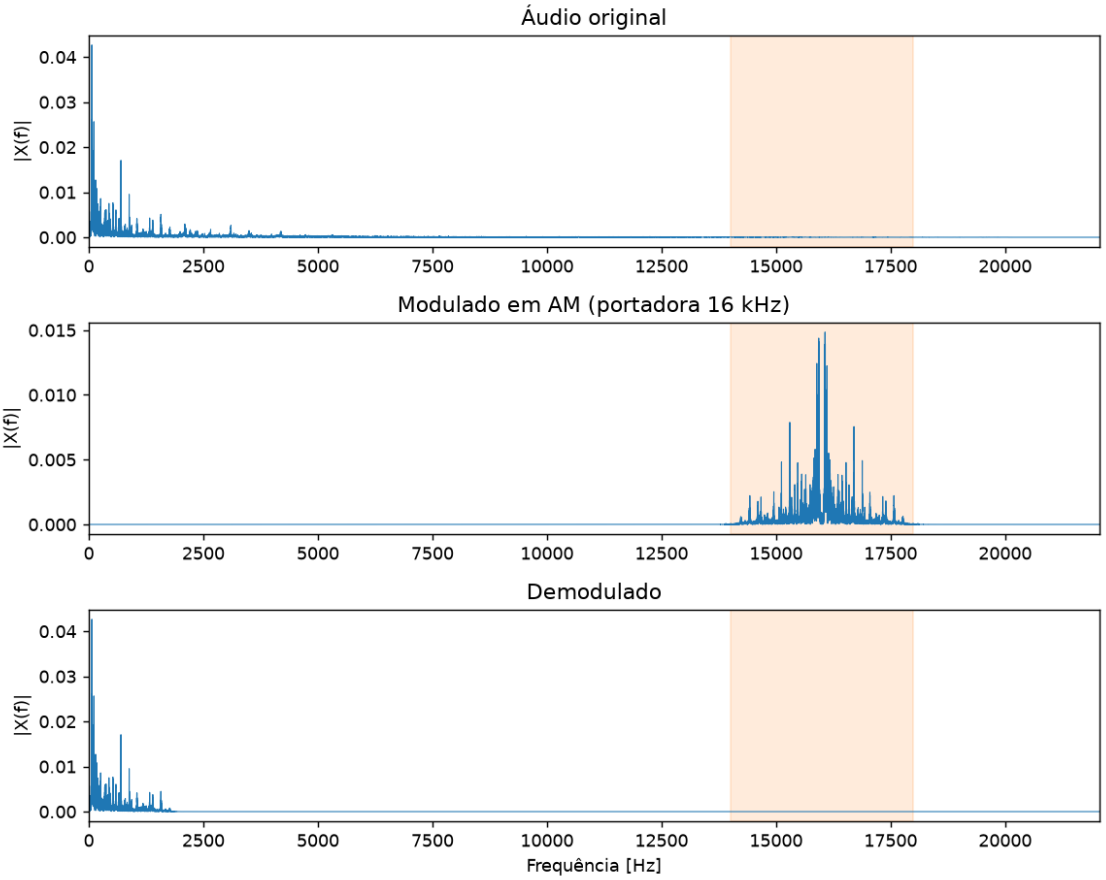
Exercício 5 – Modular e demodular um áudio

Usa o mesmo *audio1.wav* do projeto. Cada item do enunciado está marcado no código:

```
audio = carrega_audio(1)
t = np.arange(len(audio)) / FS
c = np.cos(2 * np.pi * 16000 * t)       # 2) portadora com o mesmo nº de pontos
m = passa_baixa(audio, 1800)            # 3) trata o sinal: limita a 1,8 kHz
s = m * c                               # 3) modula
d = 2 * passa_baixa(s * c, 1800)        # 6) demodula: x portadora + passa-baixa
```

1) A FFT do original mostra que a voz ocupa de ~50 Hz até alguns kHz (bem mais que 2 kHz). **3)** A faixa 14–18 kHz deixa só 2 kHz para cada lado da portadora de 16 kHz, então limitamos o áudio em 1,8 kHz (2 kHz menos uma margem para a queda do filtro, que não é vertical). **4)** O modulado fica todo dentro de 14–18 kHz: o programa mede só –47 dB de energia fora da faixa (0,002%). **5)** Ao ouvir, o modulado é um chiado agudo,

porque só tem frequências entre 14 e 18 kHz. **6–8)** Multiplicando de novo pela portadora e filtrando, a FFT do demodulado é igual à do original até 1,8 kHz, e o áudio volta a ser a voz, um pouco mais abafada.



FFT do Exercício 5: original, modulado (faixa 14–18 kHz sombreada) e demodulado.

Exercício 6 – Taxa do 64-QAM

64 combinações (8 fases × 8 amplitudes) → $\log_2(64)$ = **6 bits por símbolo**. Um símbolo a cada 2 períodos de 2,4 GHz → $2{,}4 \cdot 10^9 / 2 = 1{,}2 \cdot 10^9$ símbolos/s.
Taxa = $6 \times 1{,}2 \cdot 10^9$ = **7,2 Gbit/s** (900 MB/s).

Exercício 7 – Tempo para 480 MB em 256-QAM

256 combinações → **8 bits por símbolo**. Um símbolo a cada 10 períodos → $2{,}4 \cdot 10^8$ símbolos/s → taxa = $8 \times 2{,}4 \cdot 10^8 = 1{,}92$ Gbit/s.
480 MB = $480 \cdot 10^6 \times 8 = 3{,}84 \cdot 10^9$ bits → tempo = $3{,}84 \cdot 10^9 / 1{,}92 \cdot 10^9$ = **2 s** (se MB = 2^{20} bytes, ≈ 2,1 s).

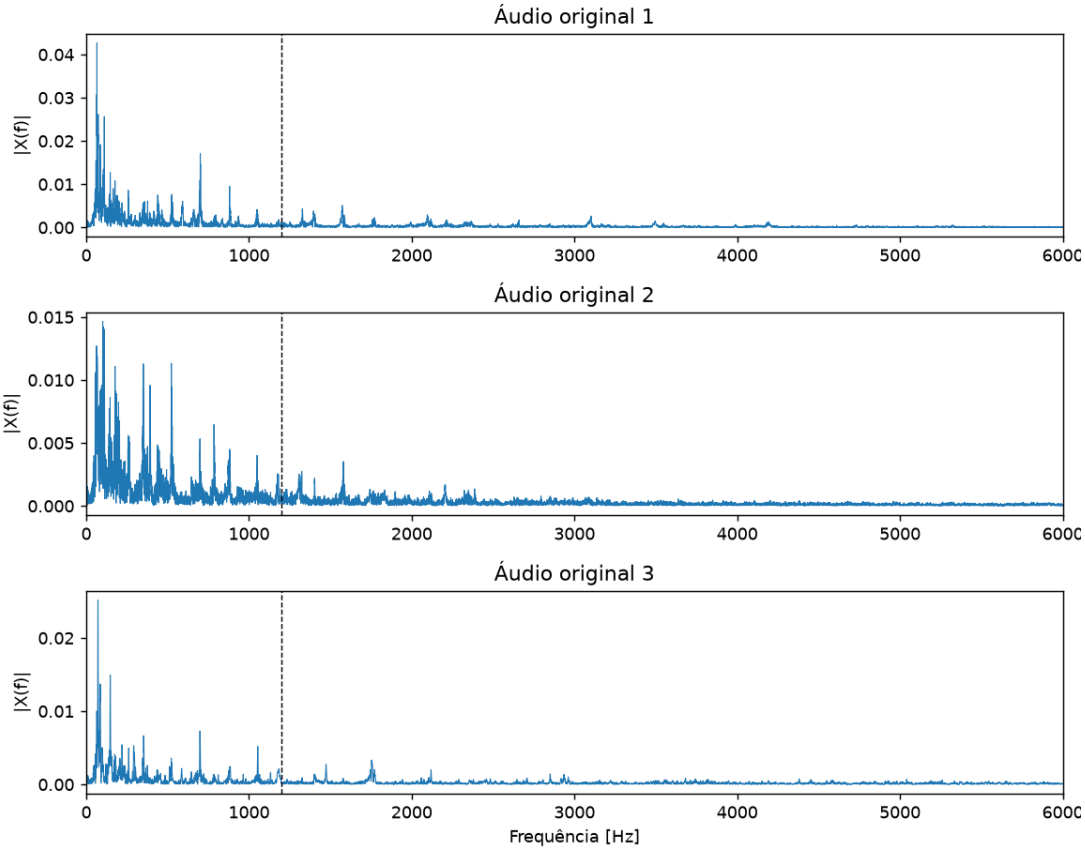
3. Projeto 8 (projeto8_am.py)

3.1 Parâmetros

```
FS = 44100
DURACAO_GRAVACAO = 4                                        # segundos por áudio
FAIXAS = [(9000, 12000), (12000, 15000), (15000, 18000)]  # uma faixa por estação
F_MSG = 1200
ORDEM = 10
```

Cada faixa tem 3 kHz de largura e a portadora fica no centro dela (10,5, 13,5 e 16,5 kHz). Como o AM por multiplicação ocupa $f_c \pm f_m$, a mensagem pode ter no máximo 1,5 kHz. Escolhemos cortar em **F_MSG = 1,2 kHz**: os 300 Hz que sobram de cada lado são a margem para a queda do filtro, já que as faixas são vizinhas e qualquer vazamento cairia direto na estação ao lado. A voz até 1,2 kHz continua inteligível (soa como um rádio

AM de verdade, meio abafado).



Espectro dos 3 áudios (0–6 kHz). A voz vai bem além de 1,2 kHz (tracejado), por isso o passa-baixa antes de modular é obrigatório.

3.2 Filtros: passa_baixa e passa_faixa

```
def passa_baixa(x, fc, fs=FS):
    return sosfiltfilt(butter(ORDEM, fc, btype="low", fs=fs, output="sos"), x)

def passa_faixa(x, f1, f2, fs=FS):
    return sosfiltfilt(butter(ORDEM, [f1, f2], btype="band", fs=fs, output="sos"), x)
```

São filtros Butterworth, como os do Projeto 7, só que de ordem 10 (o de 2ª ordem do Projeto 7 cai muito devagar para separar faixas encostadas umas nas outras). Três detalhes:

• **output="sos"**: o filtro sai como uma *cascata de filtros de 2ª ordem*, a mesma ideia do equalizador do Projeto 7 (vários G(Z) de 2ª ordem em série). Na forma (b, a) de um polinômio só, o denominador do passa-baixa de ordem 10 tem coeficientes de até ~85 com sinais alternados, e qualquer arredondamento pesa (é o mesmo tipo de problema que vimos no Projeto 7 com o np.polymul). Testamos: com estes cortes a forma (b, a) ainda funciona (diferença de ~10^{-6}), mas a cascata é a forma recomendada pelo scipy e não depende de sorte.

• **sosfiltfilt**: aplica o filtro para frente e depois para trás no sinal. O atraso de fase da ida é cancelado na volta (fase zero), então o áudio demodulado sai alinhado com o original, e a atenuação em dB dobra. Dá para fazer isso porque temos o áudio inteiro gravado.

• **fs=fs**: a frequência de corte é passada direto em Hz, sem dividir pela de Nyquist na mão.

3.3 carrega_audio, toca e salva

Mesma ideia do Projeto 7: se *audio{i}.wav* não existe, grava com `sd.rec` e salva; depois lê o arquivo, confere que está a 44100 Hz (se não estiver, o programa avisa em vez de modular com a frequência errada), converte estéreo para mono e normaliza para [–1, 1]. *toca* e *salva* normalizam o sinal antes de tocar/gravar para não saturar.

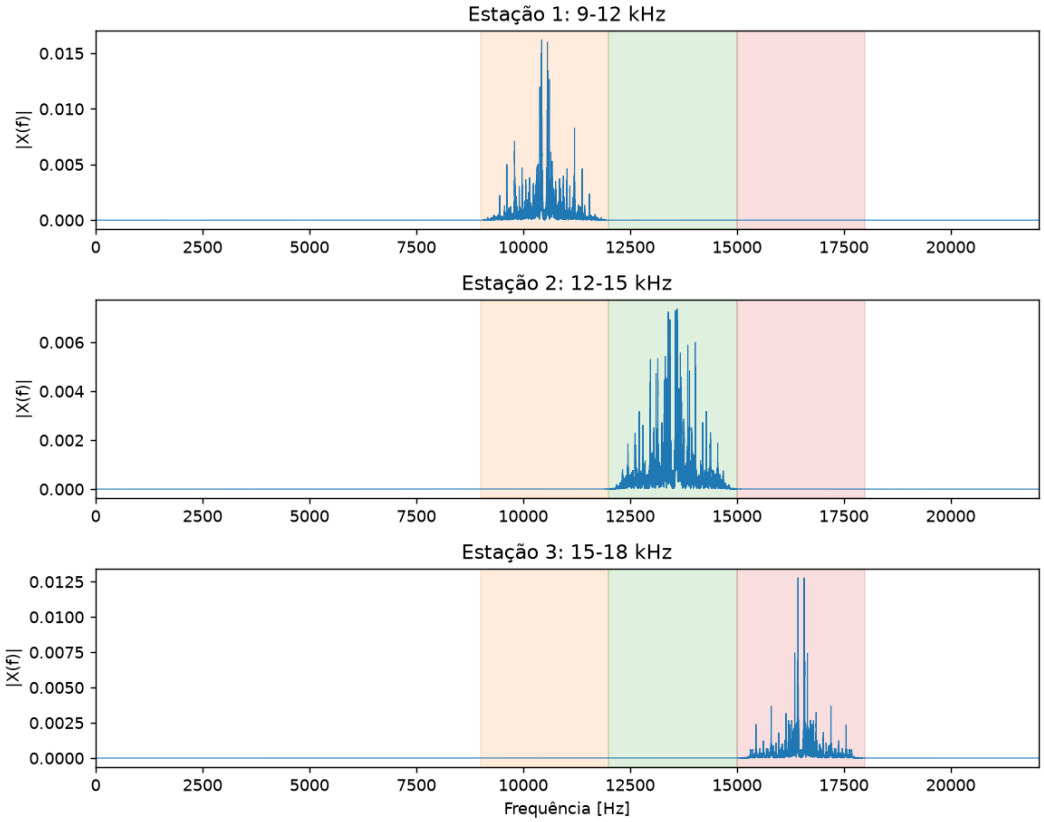
3.4 transmite – lado emissor

```
def transmite(audios):
    n = max(len(a) for a in audios)
    t = np.arange(n) / FS
    modulados = []
    for audio, (f1, f2) in zip(audios, FAIXAS):
        m = passa_baixa(np.pad(audio, (0, n - len(audio))), F_MSG)
        fc = (f1 + f2) / 2
        modulados.append(m * np.cos(2 * np.pi * fc * t))
    return modulados, sum(modulados)
```

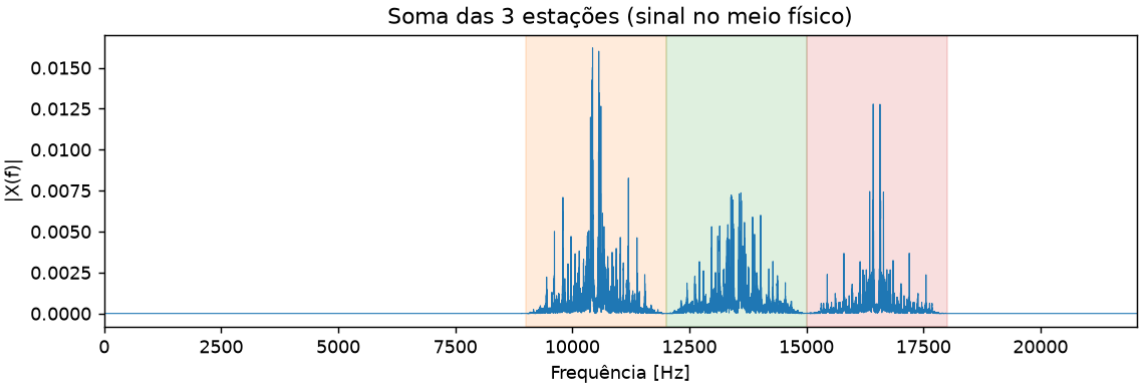
Para somar os 3 sinais eles precisam ter o mesmo número de pontos, então os áudios mais curtos são completados com zeros (`np.pad`) até o tamanho do maior. Para cada estação: limita a banda da mensagem em 1,2 kHz, calcula a portadora no centro da faixa e multiplica (modulação AM). A soma dos 3 modulados é o sinal "no ar", que todos os receptores captam.

Verificação de que ninguém invadiu a faixa do outro: a função *energia_fora* soma o $|X(f)|^2$ da FFT fora da faixa e divide pelo total. Resultado com os áudios de teste:

Estação	Faixa	Portadora	Energia fora da faixa
1	9 – 12 kHz	10,5 kHz	–51,9 dB (0,0006%)
2	12 – 15 kHz	13,5 kHz	–44,5 dB (0,004%)
3	15 – 18 kHz	16,5 kHz	–46,2 dB (0,002%)



FFT dos 3 sinais modulados. Cada um fica só na sua faixa (as cores sombreadas são as 3 faixas).



FFT da soma das 3 estações: o sinal presente no meio físico.

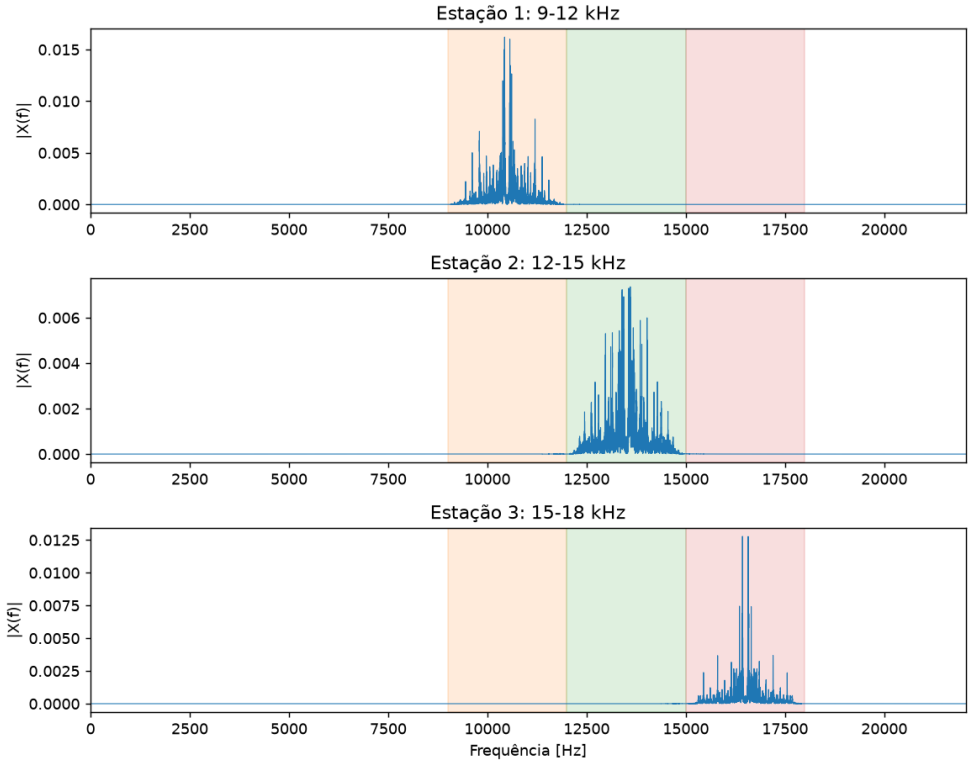
3.5 recebe – lado receptor

```
def recebe(canal):
    t = np.arange(len(canal)) / FS
    extraidos, demodulados = [], []
    for f1, f2 in FAIXAS:
        s = passa_faixa(canal, f1, f2)              # "sintoniza" a estação
        fc = (f1 + f2) / 2
        extraidos.append(s)
        demodulados.append(2 * passa_baixa(s * np.cos(2 * np.pi * fc * t), F_MSG))
    return extraidos, demodulados
```

Para cada estação, o passa-faixa deixa passar só a sua faixa: é o que o botão de sintonia de um rádio faz. Depois vem a demodulação síncrona do enunciado: multiplicar de novo pela mesma portadora (mesma frequência e fase, porque usamos o mesmo vetor de tempo). Como $\cos^2(x) = ½ + ½\cdot\cos(2x)$:

$$s(t)\cdot\cos(2\pi f_c t) = m(t)\cdot\cos^2(2\pi f_c t) = \mathbf{½\cdot m(t)} + ½\cdot m(t)\cdot\cos(2\pi\cdot 2f_c\cdot t)$$

O primeiro termo é a informação (em 0–1,2 kHz); o segundo está em torno de $2f_c$ (21, 27 ou 33 kHz). O passa-baixa de 1,2 kHz remove o segundo e o fator 2 desfaz o ½. Detalhe: 27 e 33 kHz passam de $f_s/2$ = 22,05 kHz, então aparecem rebatidos (aliasing) em 44,1 – 27 = 17,1 kHz e 44,1 – 33 = 11,1 kHz, que continuam longe de 1,2 kHz e são removidos pelo mesmo filtro.



FFT dos 3 sinais extraídos do sinal resultante pelo receptor (após o passa-faixa).

3.6 main – o que é apresentado

1. carrega os 3 áudios e **toca os 3 originais**;
2. modula e soma (*transmite*) e imprime a energia fora da faixa de cada estação;
3. separa e demodula (*recebe*) e salva *demodulado1.wav*, *demodulado2.wav* e *demodulado3.wav*;
4. abre 3 janelas: **FFT dos 3 modulados**, **FFT da soma** e **FFT dos 3 extraídos**;
5. ao fechar as janelas, **toca os 3 áudios demodulados**.

Uma aplicação só simula emissor e receptor, como o enunciado permite. Para cada FFT usamos a função *plota_fft*, que sombreia as 3 faixas com cores diferentes para dar para ver de relance se algo saiu da sua.

3.7 Validação

Autoteste (`python projeto8_am.py teste`, não precisa de microfone): coloca tons diferentes em cada estação (300, 500 e 800 Hz), passa pela cadeia inteira e verifica que (a) cada modulado tem menos de 0,01% (–40 dB) da energia fora da faixa e (b) cada demodulado volta igual ao seu tom com erro máximo menor que 1%. Como os 3 tons estão no canal ao mesmo tempo, qualquer vazamento de uma estação na outra apareceria como erro. Tons perto de 1,2 kHz voltam com alguns por cento a menos de amplitude, porque o filtro já começa a cair antes do corte.

Com áudio: comparando cada demodulado com o seu áudio original (já filtrado em 1,2 kHz), a correlação foi 0,9998, 0,9995 e 0,9995 para as estações 1, 2 e 3: os áudios foram recuperados e separados.

Observação: os gráficos deste PDF foram gerados com 3 trechos do audio_teste.wav do Projeto 7. Com os áudios gravados por vocês, os espectros mudam, mas cada estação continua só na sua faixa.